Software Maintenance Using LLMs

Laboratory Report

Student ID:	BSSE1431
Course:	Software Maintenance
Date:	May 2026
Tools Used:	Anthropic Claude Sonnet 4 (claude-sonnet-4-20250514), Google Gemini 2.0 Flash
Project:	Paint Application — Java Swing (edu.cmu.hcii.paint)

1. Introduction

This report documents the software maintenance activities performed on a Java Swing paint application. Five maintenance tasks were completed: fixing a scroll-bar rendering defect (SCROLL), correcting a color-selection bug preventing yellow from appearing (YELLOW), repairing intermittent undo functionality (UNDO), implementing a functional line-drawing tool (LINE), and adding stroke-thickness control for the pencil, eraser, and line tools (THICKNESS).

For each task, prompts were submitted to two LLM models — Anthropic Claude Sonnet 4 and Google Gemini 2.0 Flash — and their solutions were compared. The best solution was then applied to the codebase and verified.

2. Task 1 — SCROLL

Complaint / Request

Users complained that scroll bars do not always appear after painting outside the canvas boundaries, and when they do appear the canvas does not look correct — the content is clipped or offset.

Prompt Used

The following prompt was submitted to both models:

```
"In PaintCanvas.java the canvas is wrapped in a JScrollPane. Users report that
scrollbars do not always appear when content is painted outside the visible area, and
when they do appear the canvas looks wrong. Look at the paintComponent method and the
fillRect call. Identify the bug and provide the corrected Java code."
```

LLM Model

Primary model:	Anthropic Claude Sonnet 4 (claude-sonnet-4-20250514)
Secondary model:	Google Gemini 2.0 Flash

Why the Issue Appeared (per LLM)

Claude identified a typo on line 24 of PaintCanvas.java. The background fill call reads:

```
g.fillRect((int)clipBounds.getX(), (int)clipBounds.getX(), ...);
```

The second argument uses getX() instead of getY(). Because the Y origin of the clip rectangle is wrong, the white background is drawn at an incorrect vertical position. This leaves areas of the canvas unpainted, giving the appearance of a corrupted canvas. The scroll pane itself uses the component's preferred size to determine when scrollbars are needed; because the canvas preferred size is fixed at construction time and never updated as strokes extend beyond the initial bounds, the scroll pane has no way of knowing that scrollbars should appear.

Gemini independently reached the same conclusion regarding the getX()/getY() typo. It additionally noted that the preferred size of the canvas is never enlarged, but did not provide code to address that

second aspect.

How the LLM Solved the Issue

Claude's fix corrects the typo and adds logic to expand the canvas preferred size whenever a new paint object is added. The corrected fillRect call is:

```
g.fillRect((int)clipBounds.getX(), (int)clipBounds.getY(),
(int)clipBounds.getWidth(), (int)clipBounds.getHeight());
```

The addPaintObject method was also extended to revalidate the scroll pane after updating the preferred size, ensuring scrollbars appear when needed.

Where the Fix Was Applied

File:	PaintCanvas.java
Method:	paintComponent() — fillRect call
Lines changed:	1 line corrected (getX() → getY() for the Y argument)
Additional change:	addPaintObject() extended to update preferred size (~4 lines)

Testing and Verification

After applying the fix the application was compiled and run. A stroke was drawn from within the canvas beyond its right and bottom edges. The scroll pane correctly revealed scrollbars and scrolling to any position showed a clean white background with strokes rendered at their correct coordinates.

Comparison Between Models

Criterion	Claude Sonnet 4	Gemini 2.0 Flash
Root cause identified	Yes — getX()/getY() typo	Yes — same typo
Scroll bar fix included	Yes — preferred size update	Partial — mentioned but no code
Code quality	Clean, commented	Correct but no scroll fix
Lines of fix	~5	1

Best Solution

Claude's solution is preferred because it addresses both the rendering corruption and the missing scrollbar behaviour in a single patch.

3. Task 2 — YELLOW

Complaint / Request

Users complained that adjusting the colour sliders to produce yellow (Red = 255, Green = 255, Blue = 0) has no effect — the paint stroke remains green or some other colour rather than yellow.

Prompt Used

```
"In PaintWindow.java the colour is constructed from three sliders (rSlider, gSlider, bSlider). Users report they cannot select yellow. Inspect the colorChangeListener and the Color constructor call. Find the bug and provide the corrected code."
```

LLM Model

Primary model:	Anthropic Claude Sonnet 4
Secondary model:	Google Gemini 2.0 Flash

Why the Issue Appeared (per LLM)

Both models immediately spotted the bug in PaintWindow.java inside the colorChangeListener. The Color constructor is called as:

```
new Color(rSlider.getValue(), gSlider.getValue(), gSlider.getValue())
```

The third argument reads gSlider.getValue() instead of bSlider.getValue(). Consequently the blue channel always mirrors the green channel. Yellow (R=255, G=255, B=0) cannot be produced because the blue value is forced to equal green. Any colour that requires a blue value different from the green value will be wrong.

How the LLM Solved the Issue

The fix is a one-character identifier change: replace the second gSlider with bSlider in the Color constructor:

```
new Color(rSlider.getValue(), gSlider.getValue(), bSlider.getValue())
```

Where the Fix Was Applied

File:	PaintWindow.java
Method:	colorChangeListener (anonymous ChangeListener)
Lines changed:	1

Testing and Verification

After the fix, the Red and Green sliders were moved to maximum and the Blue slider to zero. The preview swatch turned yellow and a stroke drawn on the canvas appeared in yellow. Other colours (cyan, magenta, white) were also verified to confirm the blue channel now behaves independently.

Comparison Between Models

Criterion	Claude Sonnet 4	Gemini 2.0 Flash
Root cause identified	Yes	Yes
Fix accuracy	Correct	Correct
Additional comments	Explained all colours affected	Focused on yellow only
Lines of fix	1	1

Best Solution

Both models produced equivalent fixes. Claude's explanation was slightly more thorough in noting the broader impact on all colours where blue diverges from green.

4. Task 3 — UNDO

Complaint / Request

Users complained that the 'Undo my last stroke' button does not always work. After pressing undo, the canvas does not visually update; the undone stroke remains on screen.

Prompt Used

```
"In PaintCanvas.java the undo() method removes the last history snapshot but the
canvas does not visually update. Examine the undo() method and the repaint() calls in
the class. Identify the bug and provide the corrected code."
```

LLM Model

Primary model:	Anthropic Claude Sonnet 4
Secondary model:	Google Gemini 2.0 Flash

Why the Issue Appeared (per LLM)

Claude identified that the undo() method in PaintCanvas.java is missing a call to repaint() after restoring the previous state. The method correctly reassigns paintObjects to the last history entry and removes that entry from history, but never asks Swing to redraw the component. Because repaint() is not called, the on-screen pixels are not refreshed and the user sees no change.

Gemini reached the same conclusion. It also pointed out that there is a subtle aliasing issue: the history stores references to Vector snapshots, so if the same reference is mutated after being stored the history entry is corrupted. It recommended storing a defensive copy, which is already done in addPaintObject via new Vector(paintObjects).

How the LLM Solved the Issue

The fix adds a single repaint() call at the end of the undo() method:

```
public void undo() { paintObjects = (Vector)history.lastElement();
history.removeElement(history.lastElement()); repaint(); // <-- added }
```

Where the Fix Was Applied

File:	PaintCanvas.java
Method:	undo()
Lines changed:	1 line added

Testing and Verification

Several strokes were drawn on the canvas. Each press of the Undo button immediately removed the most recent stroke from the display. Repeated undo operations continued to work correctly until the canvas was empty, at which point the undo button was correctly disabled (existing logic in PaintWindow.undo()).

Comparison Between Models

Criterion	Claude Sonnet 4	Gemini 2.0 Flash
Root cause identified	Missing repaint()	Missing repaint()
Additional insight	Clean explanation	Noted reference aliasing risk
Fix lines	1	1
Fix quality	Correct and minimal	Correct with extra context

Best Solution

Both models are equivalent. Gemini's additional observation about aliasing is academically interesting, though it is not a live bug in the current code because addPaintObject already copies the vector.

5. Task 4 — LINE

Complaint / Request

Users requested a line-drawing tool. The UI already contains a 'Line' radio button, but selecting it and dragging has no effect because no corresponding action or paint class has been implemented.

Prompt Used

```
"The Paint application has a Line radio button in PaintWindow.java but it has no associated action and no LinePaint class exists. Implement: (1) a LinePaint.java class extending PaintObject that draws a straight line between the first and last points in the define() array, and (2) an action in Actions.java that switches to LinePaint. Also wire the radio button to the action in PaintWindow.java. Provide complete corrected code for all changed files."
```

LLM Model

Primary model:	Anthropic Claude Sonnet 4
Secondary model:	Google Gemini 2.0 Flash

Why the Issue Appeared (per LLM)

Claude noted that the lineButton radio button in PaintWindow.java was created with a plain string label ("Line") rather than an Action, so it was never connected to any behaviour. There is no lineAction in Actions.java and no LinePaint class in the package. The feature was scaffolded in the UI but never implemented in the model or controller layers.

How the LLM Solved the Issue

Claude produced a complete LinePaint class. The define() method stores only the first and last provided points, and paint() calls Graphics2D.drawLine() between them. getBoundingBox() returns the rectangle enclosing the two endpoints. A lineAction was added to Actions.java, and the lineButton in PaintWindow.java was changed to use lineButton = new JRadioButton(actions.lineAction) so it participates in the tool selection framework consistently with the pencil and eraser buttons.

LinePaint.java (new file — key methods):

```
public void define(Point[] points) { start = points[0]; end = points[points.length - 1]; } public void paint(Graphics2D g) { Stroke old = g.getStroke(); g.setStroke(new BasicStroke(thickness)); g.setColor(color); g.drawLine((int)start.getX(), (int)start.getY(), (int)end.getX(), (int)end.getY()); g.setStroke(old); }
```

Where the Fix Was Applied

New file:	LinePaint.java (~45 lines)
Modified file:	Actions.java — lineAction added (~8 lines added)
Modified file:	PaintWindow.java — lineButton wired to action (1 line changed)
Total lines changed:	~54 lines (45 new + 9 modified)

Testing and Verification

The Line radio button was selected. Pressing and dragging the mouse drew a straight line from the press point to the release point. During the drag, a preview line was visible, consistent with how the pencil tool shows a preview stroke. Undo correctly removed the most recently drawn line. The line respected the currently selected colour.

Comparison Between Models

Criterion	Claude Sonnet 4	Gemini 2.0 Flash
LinePaint class complete	Yes, all abstract methods	Yes, all abstract methods
Action wired correctly	Yes	Yes
Preview during drag	Yes (uses temporaryObject)	Yes
getBoundingBox()	Correct min/max logic	Correct
Code style	Consistent with codebase	Slightly verbose

Best Solution

Both models produced working solutions. Claude's output matched the code style of the existing codebase more closely, making it the preferred choice.

6. Task 5 — THICKNESS

Complaint / Request

Users requested the ability to control the stroke thickness for the pencil, eraser, and line tools. Currently the pencil and line tools are hard-coded to thickness 5 and the eraser is hard-coded to thickness 25.

Prompt Used

```
"In PaintWindow.java, objectConstructor.setThickness(5) is the only thickness setting.
Users want a UI control to change the stroke thickness for the pencil and line tools
(the eraser should remain fixed or optionally also be adjustable). Add a JSlider
labelled Thickness to the control panel, ranging from 1 to 30, with an initial value
of 5. Wire it so that moving the slider calls objectConstructor.setThickness().
Provide the modified PaintWindow.java."
```

LLM Model

Primary model:	Anthropic Claude Sonnet 4
Secondary model:	Google Gemini 2.0 Flash

Why the Issue Appeared (per LLM)

Claude explained that the thickness is set once at startup (objectConstructor.setThickness(5)) and there is no UI element to modify it at runtime. The PaintObjectConstructor class already stores a thickness field and passes it to each new PaintObject via setThickness(), so the plumbing exists — only the UI control is missing.

How the LLM Solved the Issue

Claude added a JSlider (range 1–30, initial value 5) with a JLabel to the control panel in PaintWindow.java. A ChangeListener on the slider calls objectConstructor.setThickness(thicknessSlider.getValue()) whenever the slider is moved. The slider is placed in its own JPanel and inserted into the existing controlPanel using the same GridBagLayout constraints as the other panels.

Key addition to PaintWindow.java:

```
thicknessSlider = new JSlider(1, 30, 5); thicknessSlider.setOpaque(false);
thicknessSlider.addChangeListener(e -> {
objectConstructor.setThickness(thicknessSlider.getValue()); }); thicknessPanel = new
JPanel(new FlowLayout()); thicknessPanel.setOpaque(false); thicknessPanel.add(new
JLabel("Thickness")); thicknessPanel.add(thicknessSlider);
controlPanel.add(thicknessPanel);
```

Gemini's solution was functionally identical but used an older anonymous inner class syntax instead of a lambda, and added tick marks to the slider for visual feedback.

Where the Fix Was Applied

File:	PaintWindow.java

Changes:	New JSlider field declaration, panel construction, ChangeListener wiring
Lines changed:	~12 lines added

Testing and Verification

With the pencil tool selected, the thickness slider was moved from 1 to 20. Strokes drawn at each setting reflected the chosen thickness. The same was verified for the line tool after Task 4's fix was applied. The eraser, which overrides setThickness() in EraserPaint.java to always use 25, remained unaffected by the slider, consistent with its intended fixed-size behaviour.

Comparison Between Models

Criterion	Claude Sonnet 4	Gemini 2.0 Flash
UI control added	JSlider, labelled	JSlider with tick marks
Wiring to objectConstructor	Lambda ChangeListener	Anonymous inner class
Eraser behaviour preserved	Yes	Yes
Code style	Concise, consistent	Slightly more verbose
Lines added	~12	~16

Best Solution

Claude's solution is preferred for its conciseness. Gemini's tick marks are a minor UX improvement that could optionally be incorporated.

7. Overall Summary

All five maintenance tasks were successfully resolved. The table below summarises the findings across tasks.

Task	Bug / Feature	File(s) Changed	Lines Changed	Best Model
SCROLL	fillRect Y-coordinate typo; missing scroll update	PaintCanvas.java	~5	Claude
YELLOW	Blue slider referenced as Green in Color constructor	PaintWindow.java	1	Tie
UNDO	Missing repaint() call after state restoration	PaintCanvas.java	1	Tie
LINE	No LinePaint class or action existed	LinePaint.java, Actions.java, PaintWindow.java	~54	Claude
THICKNESS	No UI for runtime thickness control	PaintWindow.java	~12	Claude

General Observations

Both LLMs reliably identified the root cause of each defect on the first prompt without requiring iterative refinement. Claude Sonnet 4 consistently produced solutions that were more tightly integrated with the existing code structure and style conventions. Gemini 2.0 Flash produced correct fixes with comparable speed and occasionally offered additional observations (aliasing risk in UNDO, tick marks in THICKNESS).

For simple one-line bug fixes (YELLOW, UNDO) both models are effectively equivalent. For feature additions requiring new classes and cross-file changes (LINE, THICKNESS) Claude's output required less post-editing to integrate cleanly into the project.

Overall, the exercise demonstrates that LLMs are effective as first-line tools for diagnosing and patching maintenance defects in moderately sized Java codebases, provided the prompt clearly scopes the relevant files and methods.